



AI VIET NAM
@aivietnam.edu.vn

AI VIET NAM - AI Conquer 2026

So sánh mô hình học máy dự đoán tình trạng bệnh nhân gan: suy diễn thống kê, calibration và khả năng tái lập

Hepatic Risk Outcome Prediction

Trương Trung Tín, Lê Đức Tùng Dương, Đỗ Hồng Nhất, Đinh Văn Anh Khôi,
Nguyễn Thị Cẩm Vân



Tóm tắt nghiên cứu

Nghiên cứu dự đoán xác suất ba tình trạng của bệnh nhân gan - còn sống hoặc bị kiểm duyệt theo dõi (C), được ghép gan (CL), tử vong (D) - trên dữ liệu bảng 9.600 quan sát, chấm bằng multiclass log loss. Dữ liệu mất cân bằng nặng (lớp CL chỉ 2.6%) và có biến thiếu tới 92%. Sáu mô hình bao gồm KNN, Logistic Regression, Decision Tree, Naive Bayes, Random Forest, XGBoost - được so sánh trên cùng một bộ 5-fold phân tầng cố định (seed 42), nhờ đó điểm của hai mô hình bất kỳ ghép được theo từng fold và so bằng kiểm định ghép cặp thay vì so hai giá trị trung bình. XGBoost tốt nhất ở cả ba chỉ số: log loss 0.382 ± 0.007 , accuracy 0.850 ± 0.006 , macro-F1 0.640 ± 0.032 , kèm top-label ECE 0.0117; sau hiệu chỉnh Bonferroni, XGBoost vượt bốn mô hình, riêng với Logistic Regression chưa đủ bằng chứng. Đóng góp chính là quy trình so sánh: dùng chung fold, kèm suy diễn thống kê, đo hiệu chuẩn và một cổng tái lập chặn cứng - cổng này tái tạo kết quả với sai lệch tối đa 5.55×10^{-17} , trong khi chỉ riêng khác biệt môi trường khi chưa ghim phiên bản thư viện đã gây lệch 0.0169, lớn hơn cả hiệu ứng 0.024 cần đo giữa XGBoost và Random Forest.

Mục lục

I.	Giới thiệu	1
II.	Nghiên cứu liên quan	2
2.1	Dự đoán tiên lượng bệnh gan	2
2.2	Các mô hình học máy truyền thống	2
2.3	Ensemble cây và mô hình hiện đại cho dữ liệu bảng	3
2.4	Đánh giá xác suất và so sánh thống kê	3
2.5	Khoảng trống và định vị nghiên cứu	3
III.	Bài toán nghiên cứu	4
3.1	Định nghĩa input và output	4
3.2	Mục tiêu tối ưu và tiêu chí đánh giá	4
3.3	Câu hỏi và giả thuyết nghiên cứu	5
IV.	Phương pháp nghiên cứu	5
4.1	Pipeline dữ liệu	6
4.2	Thiết kế validation dùng chung	7
4.3	Các mô hình và quy tắc chọn cấu hình	7
4.4	Phân tích thống kê và calibration	7
V.	Dữ liệu và thiết kế thí nghiệm	8
5.1	Dataset và chia dữ liệu	8
5.2	Thiết lập huấn luyện và đánh giá	10
VI.	Kết quả thí nghiệm	10
6.1	Suy diễn thống kê	12
6.2	Calibration	12
VII.	Ablation Study và phân tích bổ sung	13
7.1	Ảnh hưởng của scaling	13
7.2	Cây đơn và ensemble	14
VIII.	Thảo luận	15
8.1	Trả lời các câu hỏi nghiên cứu	15

8.2	Đối chiếu với nghiên cứu trước	15
IX.	Phân tích lỗi và hạn chế	16
9.1	Phân tích lỗi	16
9.2	Hạn chế nghiên cứu	16
X.	Tái lập kết quả	17
10.1	Môi trường và cách chạy lại	17
10.2	Cổng tái lập	17
XI.	Kết luận và hướng phát triển	18
XII.	Tài liệu tham khảo	19
	Phụ lục	19
A	Thông tin bổ sung	19
B	Cấu hình thí nghiệm đầy đủ	20
C	Nguồn của từng kết quả	20
D	Phân công xác nhận nội dung	21

I. Giới thiệu

Dự đoán tình trạng của bệnh nhân mắc bệnh gan là một bài toán thuộc lĩnh vực Healthcare AI. Mỗi bệnh nhân có tuổi, giới tính, thời gian theo dõi, chỉ định điều trị, giai đoạn lâm sàng, các dấu hiệu như cổ trướng, gan to, phù và sao mạch, cùng nhiều chỉ số xét nghiệm. Các kết quả trình bày trong nghiên cứu này chỉ phản ánh hiệu năng của các phương pháp học máy trên bộ dữ liệu cuộc thi [AIO 2026-1] *Hepatic Risk Outcome Prediction* (Kaggle), và không được xem là công cụ chẩn đoán hay cơ sở khuyến nghị điều trị trong thực hành lâm sàng.

Cuộc thi yêu cầu ước lượng xác suất cho ba tình trạng: còn sống hoặc bị kiểm duyệt theo dõi (C), được ghép gan và bị kiểm duyệt (CL), hoặc tử vong (D). Đầu ra phải là một phân phối xác suất hợp lệ chứ không chỉ là một nhãn, vì hệ thống chấm bằng multiclass log loss - metric phạt dự đoán sai theo hàm logarit của xác suất gán cho nhãn thực, nên sai với độ tự tin cao tổn kém hơn nhiều bậc so với sai nhưng vẫn thể hiện độ bất định. Nghiên cứu vì vậy phải xem xét đồng thời khả năng phân loại, chất lượng xác suất và calibration.

Chỉ xếp hạng các mô hình theo điểm trung bình là chưa đủ. Khoảng cách điểm giữa chúng có thể chỉ đến từ một cách chia train-validation nhất định. Ngoài ra, khi đem cùng một bộ dữ liệu ra so sánh nhiều lần, càng so sánh nhiều thì càng dễ gặp một chênh lệch trông có vẻ có ý nghĩa thống kê nhưng thực ra chỉ do ngẫu nhiên. Vì vậy, nghiên cứu này không chỉ hỏi mô hình nào điểm cao nhất, mà đặt ba câu hỏi: chênh lệch giữa các mô hình có lớn hơn mức dao động giữa các fold hay không; xác suất mà mô hình tốt nhất đưa ra có đáng tin để dùng trực tiếp hay không; và mọi con số được báo cáo có dựng lại được từ artifact gốc hay không. Ba câu hỏi này là cơ sở cho các đóng góp trình bày dưới đây.

Đóng góp

Nghiên cứu có năm đóng góp chính:

- **Pipeline dữ liệu ba tầng raw $\rightarrow$ interim $\rightarrow$ processed:** điền giá trị thiếu kèm cờ *missing*, mã hoá biến phân loại, log1p cho biến lệch phải, lưu song song hai bản có và không chuẩn hoá thang đo.
- **Quy trình so sánh công bằng cho sáu mô hình (KNN, Logistic Regression, Decision Tree, Naive Bayes, Random Forest, XGBoost):** 5-fold phân tầng (seed 42) sinh một lần, ghi ra tệp và dùng chung, nên mọi cặp mô hình đều so sánh ghép cặp được trên cùng một lần chia dữ liệu.
- **Đánh giá đa chiều bằng log loss, accuracy, macro-F1, kèm 2 ablation:** chuẩn hoá thang đo lên KNN và Logistic Regression; 4 mức regularization từ cây không giới hạn độ sâu, cây max_depth=3, Random Forest đến XGBoost.
- **Suy diễn thống kê và hiệu chuẩn xác suất:** kiểm định ghép cặp có kiểm tra giả định (Shapiro–Wilk chọn giữa paired *t*-test và Wilcoxon chính xác), kèm khoảng tin cậy và hiệu chỉnh Bonferroni; mô hình tốt nhất được đo thêm top-label ECE, classwise ECE, Brier score và độ nhảy theo số bin.
- **Provenance kiểm chứng được và cổng tái lập bắt buộc:** môi trường được ghim phiên bản trong requirements.lock.txt; manifest lưu hash SHA-256 của 8 tệp đầu vào cùng

cầu hình booster của từng fold, ghi cả những siêu tham số để nguyên giá trị mặc định. Cổng tái lập đối chiếu từng chỉ số với dung sai 10^{-5} và *chặn* mọi phân tích hiệu chuẩn cho tới khi vượt qua (Mục X.).

II. Nghiên cứu liên quan

2.1 Dự đoán tiên lượng bệnh gan

Bài toán tiên lượng bệnh gan đã được nghiên cứu từ lâu trước khi xuất hiện các bộ dữ liệu phân loại hiện đại. Dickson và cộng sự dùng mô hình Cox trên dữ liệu bệnh nhân xơ gan mật nguyên phát tại Mayo Clinic để dự đoán khả năng sống còn chỉ từ tuổi, bilirubin, albumin, thời gian prothrombin và mức độ phù, không cần sinh thiết gan, và kiểm tra lại trên một nhóm bệnh nhân độc lập [1]. Nghiên cứu này cho thấy dữ liệu dạng bảng gồm thông tin lâm sàng và xét nghiệm đã đủ mang tín hiệu tiên lượng, và mô hình chỉ đáng tin khi được đánh giá trên dữ liệu chưa từng dùng để huấn luyện.

Chính tập dữ liệu Mayo Clinic giai đoạn 1974-1984 nói trên về sau được đưa vào UCI Machine Learning Repository dưới tên Cirrhosis Patient Survival Prediction, gồm 418 bệnh nhân, 17 đặc trưng và 3 tình trạng (C, CL, D) [2]. Bộ dữ liệu của AIO Challenge trên Kaggle mà nghiên cứu này sử dụng giữ nguyên cấu trúc biến và định nghĩa nhãn đó, nhưng lớn hơn khoảng 23 lần, được chia thành ba tập: train (9.600 bản ghi), validation (2.400 bản ghi) và test (10.000 bản ghi).

2.2 Các mô hình học máy truyền thống

Bốn mô hình dưới đây được chọn vì mỗi mô hình áp đặt một giả định khác nhau lên dữ liệu, và chính giả định đó quy định điểm mạnh lẫn điểm yếu của nó:

- **Logistic Regression** giả định quan hệ tuyến tính trên thang log-odds. Nhờ vậy mô hình huấn luyện nhanh và trả về xác suất trực tiếp, nhưng đánh đổi bằng việc bỏ sót quan hệ phi tuyến và tương tác giữa các biến.
- **KNN** không giả định dạng hàm mà suy ra nhãn từ các điểm lân cận. Sự linh hoạt này khiến mô hình rất nhạy với thang đo, nhiễu và lựa chọn số láng giềng k .
- **Naive Bayes** giả định các biến độc lập với nhau. Giả định đó cho tốc độ tính toán rất cao, song lại không phù hợp với những chỉ số xét nghiệm vốn có tương quan.
- **Decision Tree** giả định dữ liệu tách được bằng những ngưỡng liên tiếp trên từng biến. Cách chia đó nắm được quan hệ phi tuyến và cho quy tắc dễ đọc, song cây sâu dao động mạnh và gán xác suất đúng 0 hoặc 1 ở những lá chỉ có vài mẫu, khiến log loss phân kỳ nếu không chặn ngưỡng.

Ensemble mạnh hơn cả bốn mô hình trên, nhưng nghiên cứu vẫn giữ đủ chúng làm đường cơ sở vì mỗi mô hình sai ở một giả định khác nhau, chênh lệch điểm số giữa chúng cho biết giả định nào bị dữ liệu vi phạm nặng nhất.

2.3 Ensemble cây và mô hình hiện đại cho dữ liệu bảng

Hai ensemble cây dưới đây khắc phục điểm yếu của cây đơn theo hai hướng ngược nhau, và mỗi hướng kéo theo một rủi ro riêng:

- **Random Forest** - nhiều cây song song, mỗi cây học trên một mẫu bootstrap và chỉ xét một tập đặc trưng ngẫu nhiên tại mỗi nút. Breiman chỉ ra sai số phụ thuộc vào sức mạnh từng cây và mức tương quan giữa chúng, nên trung bình nhiều cây ổn định hơn một cây đơn [3]. *Rủi ro*: xác suất lấy trung bình không tự nhiên mà hiệu chuẩn tốt.
- **XGBoost** - các cây xây tuần tự, cây sau sửa lỗi còn lại của các cây trước, kèm regularization, giảm bước học và lấy mẫu ngẫu nhiên để hạn chế quá khớp [4]. Đây thường là lựa chọn phù hợp cho dữ liệu bảng có quan hệ phi tuyến, tương tác giữa biến và nhiều giá trị thiếu. *Rủi ro*: kết quả nhạy với siêu tham số, và dùng chung một bộ fold cho cả dò tham số lẫn đánh giá sẽ cho điểm lạc quan.

Deep learning nằm ngoài phạm vi: dữ liệu là bảng cỡ vừa, không có ảnh, văn bản hay chuỗi thời gian, nên thêm mạng sâu chỉ làm phiền không gian thử nghiệm.

2.4 Đánh giá xác suất và so sánh thống kê

Vì hệ thống chấm bằng log loss, phần đánh giá phải trả lời hai câu hỏi tách rời nhau: xác suất mô hình đưa ra có đáng tin không, và chênh lệch giữa các mô hình có thật không. Bốn điểm dưới đây là cơ sở cho hai câu hỏi đó:

- **Log loss và Brier score** [5] chấm trên toàn bộ phân phối xác suất. Accuracy chỉ nhìn lớp thắng nên không phân biệt được một dự đoán đúng ở mức tự tin 0.51 với 0.99.
- **Reliability diagram và ECE** đo trực tiếp khoảng cách giữa độ tự tin của mô hình và tỷ lệ đúng thực tế. Đây là phần mà log loss gộp chung vào một con số duy nhất.
- **Kết quả của Guo và cộng sự** - accuracy cao không kéo theo xác suất đáng tin [6]. Kết luận này rút ra trên mạng nơ-ron, nhưng sự tách rời giữa xếp đúng lớp và hiệu chuẩn tốt không gắn riêng với họ mô hình nào, nên mô hình dẫn đầu về log loss vẫn phải đo hiệu chuẩn bằng một chỉ số riêng.
- **Khuyến nghị của Demšar** - so sánh ghép cặp bằng kiểm định phi tham số, và hiệu chỉnh khi số phép so sánh tăng lên, vì càng so sánh nhiều thì càng dễ gặp một khác biệt giả [7].

Hai điểm đầu quy định cách đo, hai điểm sau quy định cách kết luận; cả bốn được cụ thể hoá ở Mục 4.4.

2.5 Khoảng trống và định vị nghiên cứu

Bốn nhận định ở Mục 2.1-2.4 chưa được kiểm chứng cùng lúc trên cùng một cách chia dữ liệu. Vì vậy một bảng xếp hạng theo log loss trung bình còn để ngỏ ba khoảng trống:

- **Xếp hạng có nhất quán không.** Mô hình dẫn đầu ở giá trị trung bình chưa chắc dẫn đầu ở từng fold; chênh lệch không lặp lại đều thì không kết luận được (RQ1, RQ2).
- **Chênh lệch đến từ đâu.** Thang đo với KNN và xác suất cực đoan với cây (Mục 2.2, 2.3) mới chỉ là giả thuyết, chưa được đo trực tiếp (RQ3).
- **Xác suất có đáng tin không.** Xếp đúng lớp và hiệu chuẩn tốt là hai chuyện tách rời, đúng như Mục 2.4 chỉ ra (RQ4).

Nghiên cứu không đề xuất kiến trúc mới. Đóng góp là quy trình so sánh (Mục I.): một bộ fold dùng chung cho cả sáu mô hình, và một cổng tái lập để chính các con số đó kiểm chứng lại được.

III. Bài toán nghiên cứu

3.1 Định nghĩa input và output

Bài toán là phân loại đa lớp trên dữ liệu bảng. Gọi $\mathbf{x}_i \in \mathcal{X}$ là vector đặc trưng của bệnh nhân thứ i , gồm bốn nhóm: nhân khẩu học; thời gian theo dõi, điều trị và giai đoạn lâm sàng; dấu hiệu lâm sàng; và chỉ số xét nghiệm. Cột id chỉ để định danh, không làm đặc trưng. Nhãn đích là $y_i \in \mathcal{Y} = \{C, CL, D\}$.

Mô hình $f_\theta : \mathcal{X} \rightarrow \Delta^2$ ánh xạ mỗi bệnh nhân thành một vector xác suất

$$\hat{\mathbf{p}}_i = f_\theta(\mathbf{x}_i) = (\hat{p}_{i,C}, \hat{p}_{i,CL}, \hat{p}_{i,D}),$$

trong đó $0 \leq \hat{p}_{i,k} \leq 1$ và $\hat{p}_{i,C} + \hat{p}_{i,CL} + \hat{p}_{i,D} = 1$. Thứ tự ba lớp được giữ cố định là $[C, CL, D]$ ở mọi khâu: mã hoá nhãn, gọi `predict_proba` và tính metric. Lý do là nếu đảo nhầm thứ tự cột, chương trình vẫn chạy trơn tru nhưng log loss sai toàn bộ.

3.2 Mục tiêu tối ưu và tiêu chí đánh giá

Metric chính là multiclass log loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{k \in \{C, CL, D\}} y_{ik} \log(\hat{p}_{i,k}),$$

trong đó $y_{ik} = 1$ nếu quan sát i thuộc lớp k và bằng 0 trong các trường hợp còn lại. Log loss càng thấp càng tốt. Có một tính chất được dùng lại nhiều lần trong báo cáo: khi xác suất tiến về 0 thì logarit tiến ra vô cùng, nên phần đóng góp của một quan sát vào tổng *không có giới hạn trên*. Chỉ cần mô hình gán xác suất bằng 0 cho lớp thực của vài quan sát, log loss đã tăng vọt trong khi accuracy gần như không đổi. Cơ chế này giải thích phần lớn khác biệt giữa các mô hình ở Mục VI. và Mục VII..

Accuracy là tỷ lệ dự đoán đúng, tính theo lớp có xác suất cao nhất. Macro-F1 lấy trung bình F1 của ba lớp với trọng số bằng nhau, nên phản ánh tốt hơn kết quả trên lớp CL hiếm. Hai chỉ số này chỉ bổ trợ, còn việc chọn mô hình dựa trước hết vào log loss. Mô hình có log loss thấp nhất được đo thêm ECE và Brier score, để tách khả năng xếp đúng lớp khỏi độ tin cậy của xác suất.

3.3 Câu hỏi và giả thuyết nghiên cứu

- **RQ1:** Trong sáu mô hình được khảo sát, mô hình nào đạt mean 5-fold log loss thấp nhất, đồng thời duy trì accuracy và macro-F1 cạnh tranh?
- **RQ2:** Chênh lệch log loss giữa mô hình đứng đầu và từng mô hình đối chứng có đủ bằng chứng thống kê sau khi kiểm tra giả định và hiệu chỉnh cho năm phép so sánh hay không?
- **RQ3:** Scaling ảnh hưởng ra sao đến KNN và Logistic Regression? Regularization và ensemble ảnh hưởng ra sao đến độ cực đoan của xác suất do cây sinh ra?
- **RQ4:** Xác suất out-of-fold của mô hình tốt nhất có được hiệu chuẩn tốt không, và kết luận về ECE có giữ nguyên khi đổi số bin hay không?

Giả thuyết nghiên cứu

H1 ứng với RQ1: Trong sáu mô hình khảo sát, gradient boosting có regularization đạt mean 5-fold log loss thấp nhất mà không phải đánh đổi accuracy hay macro-F1.

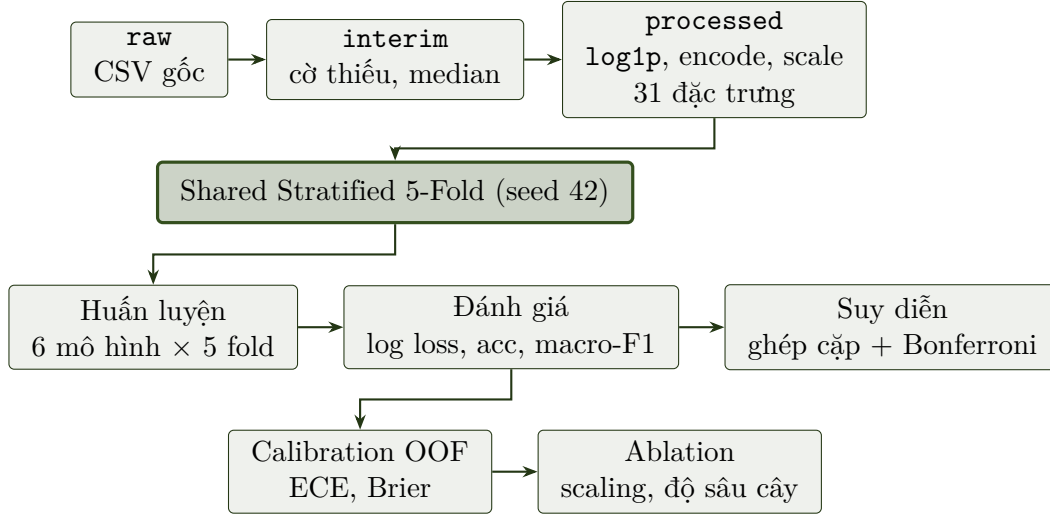
H2 ứng với RQ2: Chênh lệch log loss giữa mô hình đứng đầu và từng mô hình đối chứng đủ lớn và lặp lại đủ đều qua năm fold để vượt ngưỡng ý nghĩa sau hiệu chỉnh Bonferroni.

H3 ứng với RQ3: Scaling cải thiện KNN vì khoảng cách Euclid không còn bị chi phối bởi các đặc trưng có miền giá trị lớn. Với cây, giới hạn độ sâu và ensemble hạ log loss chủ yếu nhờ không còn gán xác suất bằng 0 cho lớp đúng, chứ không phải nhờ làm mô hình bớt tự tin nói chung.

H4 ứng với RQ4: Xác suất out-of-fold của mô hình có log loss thấp nhất lệch rất ít so với tần suất thực tế, và kết luận này không đổi khi thay đổi số bin.

IV. Phương pháp nghiên cứu

Thứ được thiết kế ở đây là **quy trình so sánh**, không phải kiến trúc mô hình. Hình 1 cho thấy vị trí của nó: khối *shared 5-fold* nằm giữa tiền xử lý và huấn luyện, buộc mọi mô hình nhận đúng cùng một cách chia dữ liệu. Nhờ vậy, với hai mô hình bất kỳ, năm điểm log loss của chúng ghép được thành năm cặp trên cùng fold để đưa vào kiểm định ghép cặp.



Hình 1: Pipeline nghiên cứu. Khối tô đậm là phần trọng tâm về phương pháp: một cách chia 5-fold cố định dùng chung cho cả sáu mô hình, làm nền cho phần kiểm định ghép cặp ở Bảng 2.

4.1 Pipeline dữ liệu

Dữ liệu đi qua ba tầng **raw** → **interim** → **processed**, mỗi tầng ghi ra file để bước sau đọc lại. Tầng **raw** giữ nguyên CSV tải về.

Tầng interim xử lý dữ liệu thiếu theo ba cách tùy mức độ:

- Edema_Status thiếu 92.01% → **loại khỏi tập đặc trưng**, chỉ giữ một cờ 0/1 đánh dấu dòng nào từng thiếu.
- Biến số thiếu nhiều → **điền median tính trên train**, mỗi biến kèm một cờ thiếu riêng để mô hình vẫn phân biệt được “thiếu” với “bằng median”.
- Biến phân loại thiếu → **gán nhãn Unknown**, không điền mode.

Ngoài ra Patient_Age_Days đổi sang tuổi theo năm; ID và fold ID không vào tập đặc trưng.

Tầng processed xử lý độ lệch và chênh thang đo:

- Sáu biến lệch phải gồm Bilirubin, Cholesterol, Copper, Alkaline Phosphatase, AST và Triglyceride được biến đổi **log1p**; thêm đặc trưng dẫn xuất là tỷ lệ Bilirubin/Albumin, cũng qua **log1p**.
- Biến phân loại mã hóa bằng **OrdinalEncoder**, giá trị chưa từng thấy ở test nhận mã **-1**.
- Biến số scale bằng **RobustScaler** vì dữ liệu nhiều outlier.

Kết quả là 31 đặc trưng, lưu thành hai bản giống hệt nhau ở mọi bước, chỉ khác ở chỗ có scale hay không. Nhờ vậy, ablation ở Mục VII. đo được riêng ảnh hưởng của scaling.

Median, encoder và scaler được fit một lần trên toàn bộ 9.600 dòng train trước khi chia fold, chứ không fit lại trong từng fold huấn luyện. Đây là một nguồn rò rỉ thông tin nhẹ, phân tích ở Mục 9.2.

4.2 Thiết kế validation dùng chung

Phân hoạch Stratified 5-Fold được sinh *đúng một lần* với `shuffle=True` và seed 42, rồi ghi ra `data/interim/fold_id.csv`. Ba nhóm mô hình không gọi lại hàm chia dữ liệu mà đọc thẳng file này, nên không thể có chuyện hai mô hình vô tình được chấm trên hai cách chia khác nhau. Đây chính là điều kiện để ghép cặp năm điểm log loss ở Mục VI.. Mỗi fold có 1.920 mẫu: fold 0 gồm 1.305 C, 50 CL, 565 D; các fold 1-4 gồm 1.304 C, 51 CL, 565 D. Toàn bộ 9.600 ID train là duy nhất. Mỗi fold, mô hình được fit lại từ đầu. Không dùng oversampling, class weighting hay dữ liệu test ở bất kỳ bước đánh giá nào.

4.3 Các mô hình và quy tắc chọn cấu hình

4.3.1 Track A: KNN và Logistic Regression

KNN được quét `n_neighbors` trong $\{3, 5, 7, 9, 11, 15, 21, 31, 41, 51\}$ trên shared 5-fold, chọn theo mean log loss và chốt $k = 51$; các tham số còn lại giữ mặc định scikit-learn (uniform weights, khoảng cách Minkowski $p = 2$). Logistic Regression không quét, dùng L2 với $C = 1$, solver `lbfgs`, `max_iter=1000` và seed 42. Cả hai chạy trên bản dữ liệu đã scale; bản không scale chỉ dùng cho ablation.

4.3.2 Track B: Decision Tree và Naive Bayes

Decision Tree được quét `max_depth` trong $\{2, 3, 4, 5, 6, 8, 10, 12, 15, 20, \text{None}\}$, chốt độ sâu 3 theo mean log loss, dùng criterion Gini, `min_samples_leaf=1`, seed 42. Mô hình nền thứ hai là GaussianNB với `var_smoothing=10-9` mặc định. Xác suất *không* bị cắt ngưỡng trước khi tính metric (chỉ còn lớp bảo vệ mặc định sẵn trong `sklearn.log_loss`), để ablation phản ánh đúng hậu quả của việc cây không giới hạn độ sâu gán xác suất bằng 0.

4.3.3 Track C: Random Forest và XGBoost

Cả hai được grid search trên số cây và độ sâu, riêng XGBoost quét thêm learning rate (không gian đầy đủ ở Bảng 3). Cấu hình chốt: Random Forest 600 cây độ sâu 14, XGBoost 400 cây độ sâu 2 với learning rate 0.15 - một bên là ensemble ít cây nhưng sâu, một bên nhiều cây nhưng nông. Cả hai đều không dùng early stopping và không đánh trọng số lớp, để chênh lệch giữa chúng phản ánh khác biệt giữa bagging và boosting, chứ không phản ánh mức can thiệp vào lớp hiếm. Sau khi chốt tham số, cả hai được chạy lại trên cùng năm fold để sinh bảng điểm cuối và dự đoán out-of-fold.

4.4 Phân tích thống kê và calibration

XGBoost được chọn làm mô hình tham chiếu vì có log loss trung bình thấp nhất. Với mỗi mô hình đối chứng, quy trình chạy đúng bốn bước:

1. **Tính năm chênh lệch ghép cặp** $d_j = \mathcal{L}_{\text{XGB},j} - \mathcal{L}_{\text{comp},j}$, trong đó $d_j < 0$ nghiêng về XGBoost.
2. **Chạy Shapiro-Wilk trên năm chênh lệch đó để chọn kiểm định chính:** $p \geq 0.05$ lấy p-value của paired t-test, $p < 0.05$ lấy p-value của exact Wilcoxon hai phía.

3. **Hiệu chỉnh Bonferroni** cho năm phép so sánh với mức ý nghĩa chung $\alpha = 0.05$, tức ngưỡng thực tế cho mỗi phép so sánh là 0.01.
4. **Ghi ra bảng cả hai p-value**, kể cả cái không được chọn, để thấy kết luận phụ thuộc nhiều hay ít vào bước 2.

Khoảng tin cậy 95% chỉ để mô tả; kết luận dựa trên kiểm định chính.

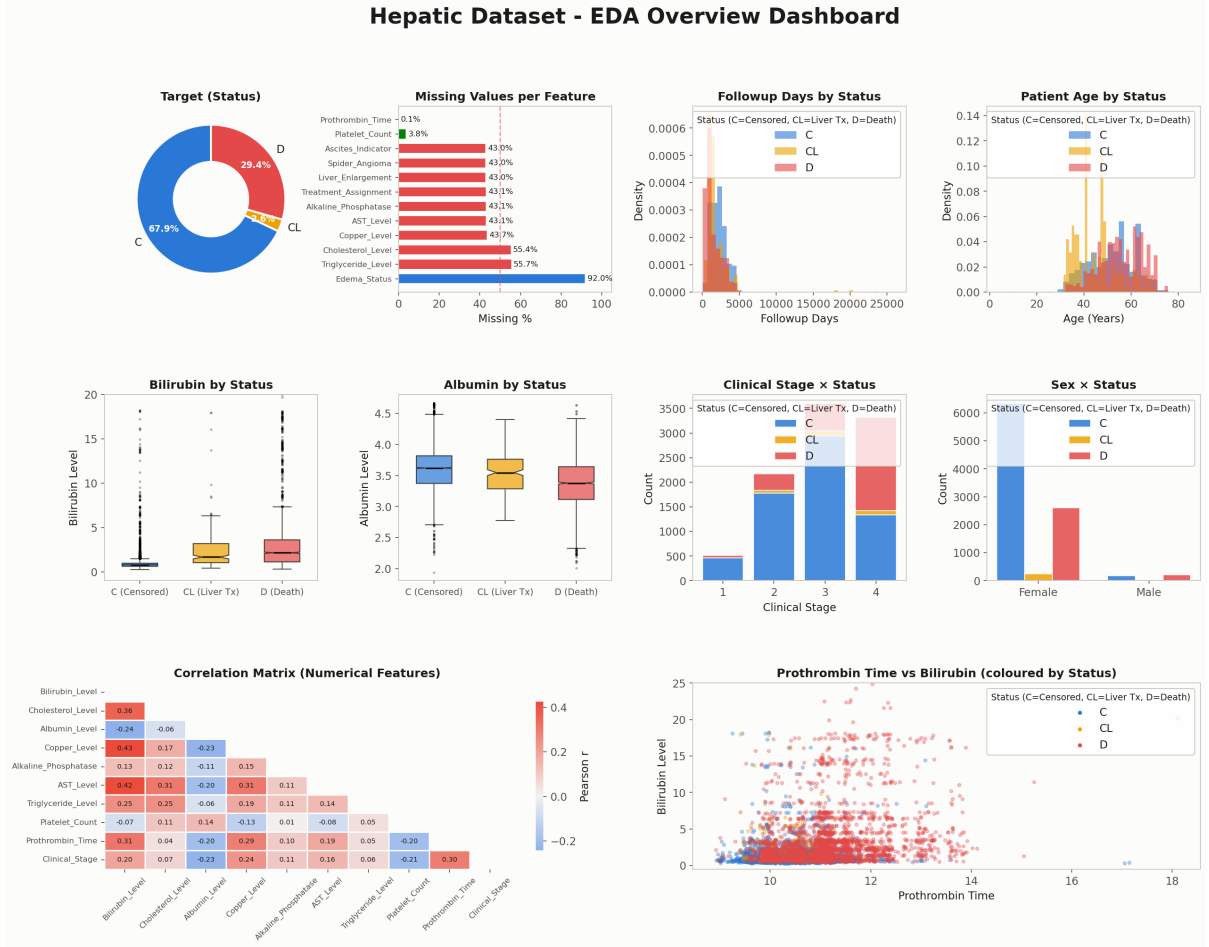
Dự đoán out-of-fold của XGBoost chỉ được dùng để đo hiệu chuẩn sau khi công tái tạo ở Mục X. xác nhận metric khớp bảng điểm trên cả năm fold. Phần này đo bằng reliability diagram, top-label ECE, classwise ECE, multiclass Brier score, kèm bảng ECE theo 5, 10, 15 và 20 bin.

V. Dữ liệu và thiết kế thí nghiệm

5.1 Dataset và chia dữ liệu

Dữ liệu tải từ trang cuộc thi AIO26 Challenge trên Kaggle gồm ba tập: `aio26_train.csv` (9.600 dòng, 18 đặc trưng, ID và nhãn), `aio26_val.csv` (2.400 dòng có nhãn) và `aio26_test.csv` (10.000 dòng không nhãn). Mọi con số trong báo cáo đều tính từ 9.600 dòng train trên bộ 5-fold dùng chung. File val không gộp vào, còn test không có nhãn nên không tham gia dò tham số hay đánh giá ở bất kỳ bước nào.

Trong train, C có 6.521 mẫu (67.93%), D có 2.825 mẫu (29.43%) và CL có 254 mẫu (2.65%). Mỗi lượt huấn luyện dùng 7.680 mẫu và đánh giá trên 1.920 mẫu còn lại.



Hình 2: Tổng quan kích thước, phân bố nhãn và tỷ lệ thiếu trên tập train 9.600 mẫu.

Hình 2 tổng hợp ba đặc điểm của bộ dữ liệu, mỗi đặc điểm ràng buộc trực tiếp một lựa chọn trong tiền xử lý hoặc tiêu chí đánh giá:

- **Nhãn mất cân bằng nghiêm trọng, tỉ lệ C:CL xấp xỉ 26:1.** Với phân bố này, một mô hình chỉ cần dự đoán thiên về lớp đa số đã đạt accuracy cao trong khi gần như bỏ qua hoàn toàn lớp hiếm, nên accuracy không còn là thước đo đáng tin và mọi bảng điểm đều phải báo cáo kèm macro-F1.
- **Dữ liệu thiếu ở mức cao.** Edema_Status thiếu 92.01%, Triglyceride và Cholesterol thiếu 55.66% và 55.35%, nhiều biến điều trị, dấu hiệu và xét nghiệm thiếu khoảng 43%. Ở ngưỡng 92%, diện khuyết gần như đồng nghĩa với sinh dữ liệu giả, nên phải cân nhắc giữa loại bỏ biến và giữ lại kèm cờ đánh dấu giá trị thiếu - căn cứ cho ba cách xử lý thiếu ở Mục 4.1.
- **Miền giá trị chênh nhau tới bốn bậc độ lớn, kèm phân phối lệch phải.** Albumin dao động trong khoảng 1.94 – 4.67, trong khi Alkaline Phosphatase trải từ 8.1 đến 19.053; Bilirubin, Copper, Cholesterol và vài xét nghiệm khác có trung bình lớn hơn trung vị nên được log1p. Nếu không chuẩn hoá, các biến có miền giá trị lớn sẽ lấn át cả khoảng cách lẫn số hạng phạt, khiến những mô hình dựa trên khoảng cách hoặc có regularization bị lệch nghiêm trọng.

Histogram của tám biến chính cho thấy train và test phân bố khá giống nhau, nhưng đây chỉ là quan sát bằng mắt; nghiên cứu không chạy kiểm định nào để đo mức lệch phân phối, nên không kết luận hai bên giống nhau.

5.2 Thiết lập huấn luyện và đánh giá

Toàn bộ thí nghiệm chạy trong môi trường đã pin ở `requirements.lock.txt` (chi tiết phiên bản ở Phụ lục B) với seed thống nhất 42.

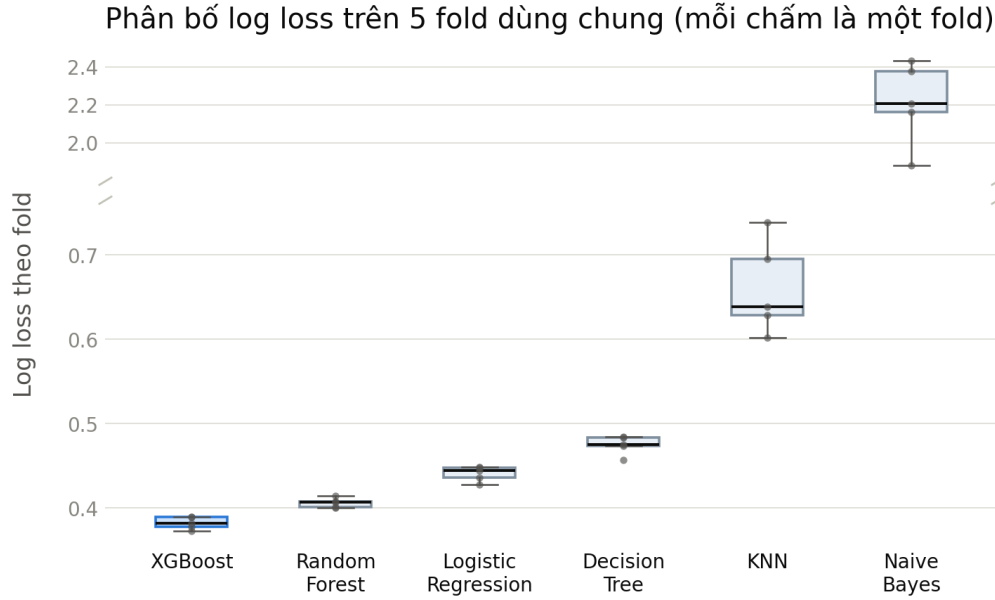
Mỗi mô hình được tạo mới và fit lại từ đầu trên từng fold huấn luyện, rồi chấm bằng một hàm metric dùng chung cho cả sáu mô hình. Hàm này luôn truyền `labels=[0,1,2]`, nhờ đó fold nào ít mẫu CL vẫn trả về đủ ba cột xác suất thay vì báo lỗi hoặc lệch cột. Log loss quyết định việc chọn cấu hình và xếp hạng; accuracy và macro-F1 chỉ báo cáo kèm. Kết quả cuối là $6 \times 5 = 30$ dòng điểm, mỗi dòng có thêm một cờ xác nhận log loss tính lại khớp với bảng điểm đã lưu. Thời gian chạy của `fit` và `predict_proba` được đo riêng.

Siêu tham số được chọn theo log loss trung bình trên chính bộ fold dùng chung, rồi hiệu năng cũng báo cáo trên bộ fold đó. Cách làm này nhất quán giữa các mô hình nhưng không phải nested CV (xem Mục 9.2). Sau khi chọn xong, tham số được chốt lại trước khi gộp bảng điểm và chạy phân tích thống kê.

VI. Kết quả thí nghiệm

Bảng 1: Hiệu năng 5-fold (trung bình $\pm$ độ lệch chuẩn).

Mô hình	Log loss $\downarrow$	Accuracy $\uparrow$	Macro-F1 $\uparrow$
XGBoost	0.382 ± 0.007	0.850 ± 0.006	0.640 ± 0.032
Random Forest	0.406 ± 0.006	0.848 ± 0.007	0.569 ± 0.008
Logistic Regression	0.441 ± 0.009	0.833 ± 0.005	0.558 ± 0.014
Decision Tree	0.474 ± 0.011	0.830 ± 0.007	0.529 ± 0.008
KNN	0.660 ± 0.055	0.814 ± 0.009	0.515 ± 0.010
Naive Bayes	2.211 ± 0.217	0.589 ± 0.030	0.466 ± 0.019



Hình 3: Phân bố log loss của sáu mô hình trên cùng năm fold; mỗi chấm là một fold. Trục y bị ngắt ở khoảng 0.8–1.8 vì không có quan sát nào rơi vào đó.

XGBoost đứng đầu cả ba chỉ số. So với Random Forest, log loss giảm từ 0.406 xuống 0.382 còn accuracy gần như không đổi (tăng 0.002). Cải thiện rõ nhất nằm ở macro-F1, từ 0.569 lên 0.640 - dấu hiệu tốt cho lớp CL hiếm, nhưng muốn kết luận riêng cho lớp CL thì phải xem confusion matrix. Logistic Regression đứng thứ ba với 0.441, cho thấy một mô hình tuyến tính đơn giản vẫn cạnh tranh được. KNN và Naive Bayes có log loss cao trong khi accuracy không tụt tương ứng, đúng như cơ chế log loss không có giới hạn trên đã nêu ở Mục 3.2.

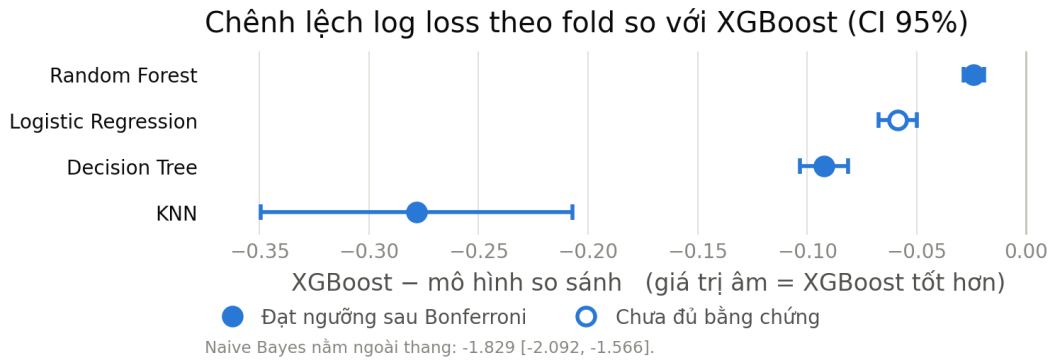
Hình 3 cho thấy điều mà giá trị trung bình che mất: thứ hạng giữ nguyên ở cả năm fold, nhưng độ ổn định thì khác hẳn nhau. Bốn mô hình đầu chỉ chênh 0.014–0.027 log loss giữa fold tốt nhất và fold tệ nhất, trong khi KNN trải 0.602–0.738 và Naive Bayes trải 1.880–2.431. Điểm của hai mô hình sau phụ thuộc nhiều vào việc rơi trúng fold nào, nên phần suy diễn so từng cặp theo fold thay vì so hai giá trị trung bình.

Về thời gian chạy, Decision Tree và Naive Bayes fit nhanh nhất, Logistic Regression cũng nhẹ; XGBoost fit khoảng 0.64 giây mỗi fold, nhanh hơn Random Forest (1.19 giây). Thời gian dự đoán của KNN không ổn định, riêng fold đầu chậm bất thường. Notebook không ghi cấu hình CPU, RAM hay GPU, nên các số này chỉ mô tả lần chạy hiện tại, không phải kết quả đo hiệu năng phần cứng.

6.1 Suy diễn thống kê

Bảng 2: So sánh XGBoost với từng mô hình đối chứng. Kết luận dựa trên kiểm định chính.

Mô hình đối chứng	$\bar{d}$	CI 95%	Shapiro p	p chỉnh	Kết luận
Random Forest	-.0240	[-.0287, -.0192]	.4627	.00075	XGB tốt hơn
Logistic Regression	-.0586	[-.0673, -.0498]	.0142	.31250	Chưa đủ bằng chứng
Decision Tree	-.0923	[-.1035, -.0812]	.5161	.00011	XGB tốt hơn
KNN	-.2783	[-.3496, -.2071]	.3438	.00205	XGB tốt hơn
Naive Bayes	-1.8287	[-2.0919, -1.5656]	.5702	.00021	XGB tốt hơn

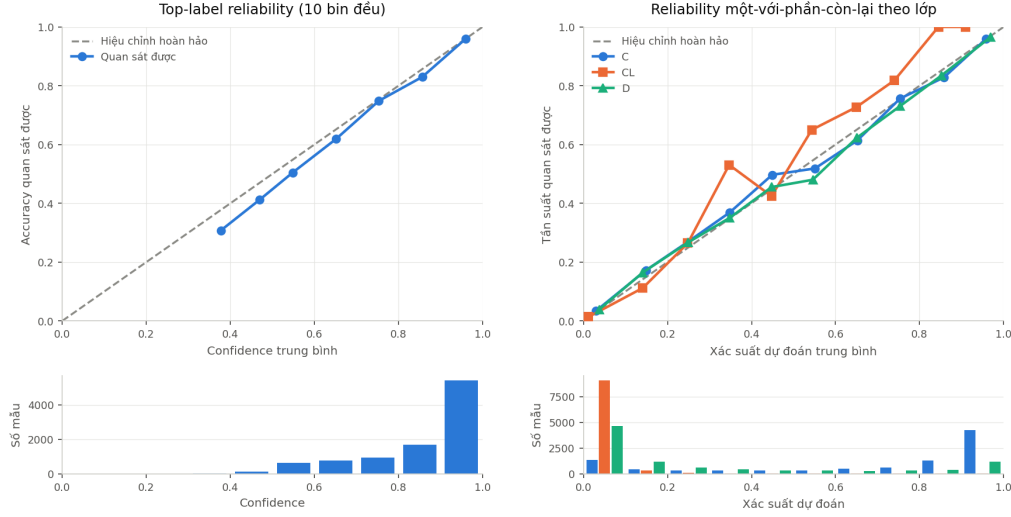


Hình 4: Chênh lệch log loss theo fold giữa XGBoost và từng mô hình đối chứng, kèm khoảng tin cậy 95%. Điểm đặc là các so sánh đạt ngưỡng sau Bonferroni; điểm rỗng là Logistic Regression, nơi khoảng tin cậy nằm trọn bên trái vạch 0 nhưng kiểm định chính (Wilcoxon) không kết luận được. Naive Bayes nằm ngoài thang của hình, số đầy đủ ở Bảng 2.

Bốn trong năm so sánh đạt ngưỡng sau hiệu chỉnh. Ngoại lệ là Logistic Regression. Shapiro-Wilk bác bỏ giả định phân phối chuẩn ($p = .0142$) nên kiểm định chính chuyển sang exact Wilcoxon. Với $n = 5$, p-value hai phía nhỏ nhất mà kiểm định này có thể trả về là .0625, tức đã vượt ngưỡng .01 *trước khi nhìn vào dữ liệu*. Vậy “chưa đủ bằng chứng” ở đây nói lên giới hạn của thiết kế chỉ năm fold, không phải nói hai mô hình ngang nhau. Khoảng tin cậy của cả năm chênh lệch đều nằm trọn dưới 0, nhưng riêng với cặp này nó chỉ để tham khảo. Ngoài ra, năm tập huấn luyện trùng lặp nhau khá nhiều, nên năm chênh lệch không phải nằm quan sát độc lập.

6.2 Calibration

Tính trên 9.600 dự đoán out-of-fold đã qua cổng tái tạo, XGBoost có top-label ECE 0.0117, classwise ECE trung bình 0.0085 và multiclass Brier score 0.2162 ở 10 bin. Đổi số bin sang 5, 15, 20 thì ECE chỉ dao động trong 0.0117-0.0144, nên kết quả này không phải nhờ chọn được một cách chia bin có lợi.



Hình 5: Reliability diagram của XGBoost trên dự đoán out-of-fold đã qua cổng tái tạo.

Hình 5 cho thấy đường reliability bám sát đường chéo ở vùng mô hình tự tin cao. Trong bảy bin có dữ liệu, tỷ lệ đúng thực tế luôn thấp hơn độ tự tin một chút, tức XGBoost tự tin hơi quá theo một hướng cố định chứ không dao động ngẫu nhiên. Mức lệch nhỏ và tập trung ở vùng ít dữ liệu: bin $[0.9, 1.0)$ chứa 5.416 mẫu (56.4%) nhưng chỉ lệch 0.00006, còn mọi mức lệch vượt 0.03 đều nằm ở các bin có độ tự tin dưới 0.7, trong đó bin $[0.3, 0.4)$ chỉ có 13 mẫu. Ba bin dưới 0.3 trống là điều tất yếu của bài toán ba lớp, vì xác suất lớn nhất luôn ít nhất bằng $1/3$.

Classwise ECE là 0.0119 cho C, 0.0038 cho CL và 0.0097 cho D. Giá trị thấp nhất thuộc về CL *không* có nghĩa lớp này được hiệu chuẩn tốt nhất. CL chỉ có 254 mẫu và 94.5% quan sát rơi vào bin xác suất $[0, 0.1)$, nên ở đây ECE chủ yếu đo xem xác suất gần 0 có khớp với tần suất gần 0 hay không - một phép đo gần như luôn đạt với lớp hiếm, dù mô hình có nhận ra CL hay không. Nghiên cứu chỉ đo hiệu chuẩn trên dự đoán out-of-fold, không áp dụng Platt, isotonic hay temperature scaling, và không kết luận gì về hiệu chuẩn trên tập test ẩn.

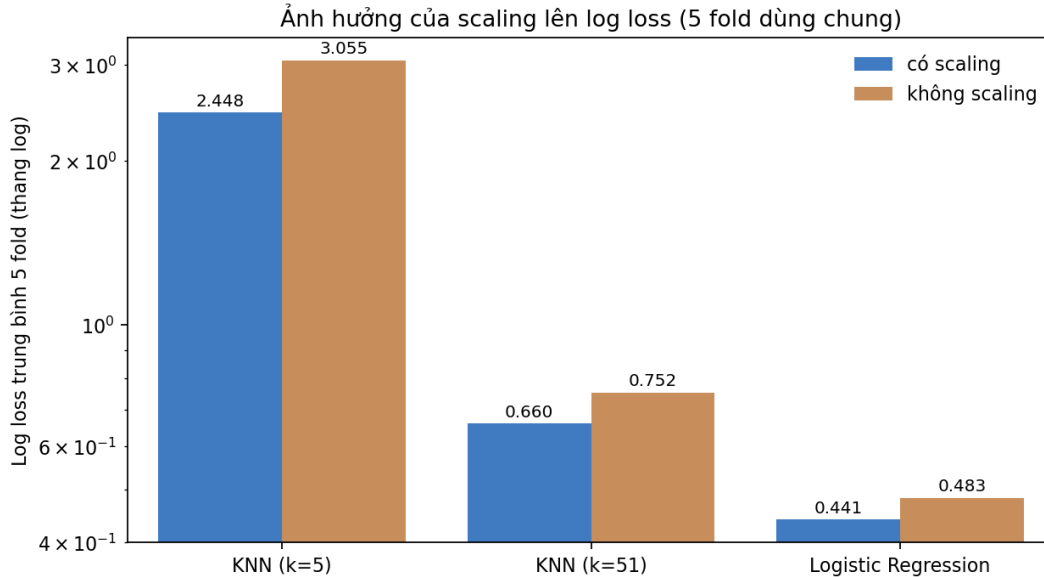
VII. Ablation Study và phân tích bổ sung

7.1 Ảnh hưởng của scaling

Ablation này chạy cùng mô hình, cùng năm fold, chỉ đổi giữa hai bản dữ liệu có và không scaling đã lưu ở Mục 4.1. Với KNN $k = 51$, scaling giảm log loss trung bình từ 0.752 xuống 0.660, tăng accuracy từ 0.780 lên 0.814 và macro-F1 từ 0.477 lên 0.515. Logistic Regression hưởng lợi ít hơn, log loss giảm từ 0.483 xuống 0.441. Chênh lệch này đúng với cách hoạt động của hai mô hình. Khoảng cách Euclid cộng bình phương sai lệch của mọi chiều, nên một biến trải bốn bậc độ lớn có thể áp đảo toàn bộ phép đo. Logistic Regression thì chỉ chịu ảnh hưởng gián tiếp, qua độ ổn định của bài toán tối ưu và qua mức phạt L2 đặt lên các hệ số khác thang đo.

Số lảng giềng tác động mạnh hơn scaling. KNN $k = 5$ có log loss trung bình 2.448 khi scale và

3.055 khi không scale, gấp khoảng bốn lần $k = 51$. Lý do là xác suất ước lượng từ năm láng giềng chỉ nhận các giá trị bội của $1/5$, nên một quan sát bị phân loại sai sẽ nhận đúng xác suất 0 cho nhãn thực và bị phạt rất nặng.



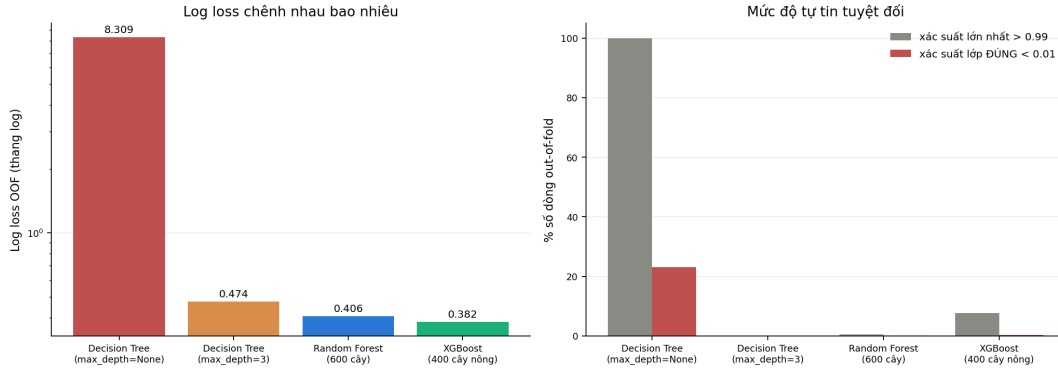
Hình 6: Ablation ảnh hưởng của scaling đối với KNN và Logistic Regression.

7.2 Cây đơn và ensemble

Ablation này so bốn mức kiểm soát độ phức tạp trên cùng dữ liệu: cây không giới hạn độ sâu, cây `max_depth=3`, Random Forest và XGBoost. Log loss giảm dần đều: $8.309 \rightarrow 0.474 \rightarrow 0.406 \rightarrow 0.382$.

Hình 7 cho thấy độ tự tin *không* đi cùng chiều với log loss: tỷ lệ mẫu có $p_{\max} > 0.99$ lần lượt là 100%, 0%, 0.6% rồi 7.7%. Trong ba mô hình đã được kiểm soát độ phức tạp, XGBoost vừa tự tin nhất vừa tốt nhất, nên độ tự tin không giải thích được chênh lệch.

Yếu tố thực sự phân biệt là xác suất nhỏ nhất từng gán cho lớp đúng. Cây không giới hạn độ sâu để giá trị này tụt xuống 0 ở 23.1% số mẫu; vì log loss không có giới hạn trên, riêng nhóm này đã đủ đẩy chỉ số lên 8.309. Ba mô hình còn lại đều giữ một mức sàn dương: 0.0085 với cây độ sâu 3, 0.0032 với Random Forest và 0.0008 với XGBoost. Nói cách khác, regularization và ensemble thẳng không phải vì làm mô hình bớt tự tin, mà vì làm nó *không bao giờ chắc chắn tuyệt đối vào một đáp án sai*. XGBoost được phép tự tin hơn vì độ tự tin đó khớp với thực tế (Mục 6.2).



Hình 7: So sánh log loss và mức độ tự tin của cây đơn với các ensemble cây.

VIII. Thảo luận

8.1 Trả lời các câu hỏi nghiên cứu

RQ1 - H1 được xác nhận. XGBoost đạt mean 5-fold log loss thấp nhất (0.382 ± 0.007) đồng thời giữ accuracy và macro-F1 cao nhất. Không có đánh đổi giữa ba chỉ số, nên kết luận không phụ thuộc vào việc chọn metric nào làm tiêu chí.

RQ2 - H2 chỉ đúng một phần. Sau Bonferroni, XGBoost tốt hơn bốn mô hình đối chứng, riêng với Logistic Regression thì chưa kết luận được. Nguyên nhân là thiết kế chỉ có năm fold (Mục VI.), không phải vì hai mô hình ngang nhau. Đây là chỗ phân biệt hai câu hỏi thường bị gộp làm một: mô hình nào điểm cao hơn, và chênh lệch đó có lặp lại đều qua các fold hay không.

RQ3 - H3 đúng ở cả hai vế. Scaling cải thiện cả KNN lẫn Logistic Regression. Với cây, vế thứ hai đúng như phát biểu và đồng thời bác bỏ một cách hiểu phổ biến: yếu tố quyết định không phải độ tự tin nói chung, mà là việc gán xác suất bằng 0 cho lớp đúng - lỗi chỉ xuất hiện ở cây không giới hạn độ sâu (Mục VII.).

RQ4 - H4 đúng ở mức tổng thể, chưa đúng ở mức từng lớp. Top-label ECE là 0.0117 và giữ nguyên khi đổi số bin, nên kết luận “hiệu chuẩn tốt” không đến từ một cách chia bin có lợi. Riêng chỉ số classwise của CL bị chi phối bởi việc lớp này quá hiếm chứ không phản ánh độ tin cậy của dự đoán, nên không dùng để kết luận.

8.2 Đối chiếu với nghiên cứu trước

Kết quả khớp với ba nhận định trong tài liệu, nhưng ở mức độ khác nhau.

Thứ nhất, ensemble cây tốt hơn cây đơn, đúng như Breiman [3] và Chen & Guestrin [4] mô tả. Tuy nhiên khoảng cách ở đây khá nhỏ: từ Random Forest sang XGBoost log loss chỉ giảm 0.024, thua xa khoảng cách giữa các ensemble với cây không giới hạn độ sâu.

Thứ hai, nhận định của Guo và cộng sự [6] lặp lại nhưng nhẹ hơn nhiều. Guo thấy mạng nơ-ron tự tin quá mức nghiêm trọng, còn XGBoost ở đây chỉ tự tin hơi quá và theo một hướng cố định, nên temperature scaling nhiều khả năng chỉ giúp thêm chút ít. Minh họa rõ nhất cho nguyên tắc “accuracy tốt không đảm bảo xác suất tốt” lại là Naive Bayes: accuracy 0.589 nhưng log loss

2.211.

Thứ ba, cảnh báo của Demšar [7] đúng, nhưng ở đây phải đọc dè dặt hơn. Demšar khuyến nghị Wilcoxon cho so sánh trên *nhiều bộ dữ liệu*, còn nghiên cứu này chỉ có một bộ dữ liệu với năm fold phụ thuộc nhau. Vì vậy các p-value ở Bảng 2 chỉ nên hiểu là dấu hiệu cho thấy chênh lệch có lặp lại qua các fold, chứ không phải bằng chứng mô hình sẽ tốt tương tự trên nhóm bệnh nhân khác.

IX. Phân tích lỗi và hạn chế

9.1 Phân tích lỗi

Lỗi của các mô hình rơi vào hai lỗi khác hẳn nhau:

- **Lớp hiếm.** CL chỉ có 254 mẫu, tức 50-51 mẫu mỗi fold đánh giá, nên mọi ước lượng riêng cho lớp này đều rất kém chắc chắn. Đó là lý do macro-F1 thấp hơn accuracy ở mọi mô hình, và cũng là lý do không thể đọc classwise ECE của CL như bằng chứng về độ tin cậy (Mục 6.2).
- **Xác suất cực đoan mà accuracy không phát hiện được.** Decision Tree không giới hạn độ sâu vẫn đạt accuracy 0.769, gần bằng nhóm dẫn đầu, nhưng log loss lên tới 8.309. KNN với k nhỏ mắc đúng lỗi này. Naive Bayes có log loss 2.211 vì giả định các biến độc lập khiến nó quá tự tin trên những xét nghiệm vốn có tương quan.

Cả ba trường hợp ở kiểu 2 cho thấy một mô hình có thể xếp đúng lớp mà vẫn ước lượng xác suất sai nghiêm trọng.

Các nhóm mô hình đều có dựng confusion matrix và bảng mức độ quan trọng của đặc trưng, trong đó Bilirubin, Prothrombin Time, Copper, Albumin và tỷ lệ Bilirubin/Albumin thường xếp cao. Nhưng chỉ số này chỉ cho biết mô hình dùng biến nào nhiều, không chứng minh quan hệ nhân quả, nên nghiên cứu không quy nguyên nhân lâm sàng cho bất kỳ dự đoán sai nào.

9.2 Hạn chế nghiên cứu

Ba giới hạn có thể khiến điểm số và p-value đẹp hơn thực tế:

- **Rò rỉ thông tin nhẹ ở tiền xử lý.** Median, encoder và scaler được fit trên toàn bộ 9.600 dòng train trước khi chia fold, nên thông tin phân phối của phần dữ liệu đánh giá đã lọt vào bước tiền xử lý. Nhân thì không bị dùng, nhưng đây vẫn là rò rỉ. Báo cáo giữ nguyên kết quả đã chốt; bản sau cần đưa các bước này vào Pipeline/ColumnTransformer để fit riêng trong từng fold.
- **Điểm số bị chọn có lợi.** Việc dò tham số và việc đánh giá dùng chung một bộ fold, thay vì tách ra bằng nested CV.
- **Không đăng ký trước.** Quy trình được chốt sau khi đã thấy kết quả sơ bộ.

Hai giới hạn còn lại thu hẹp phạm vi kết luận:

- **Quá ít fold để kiểm định.** Với $n = 5$, Shapiro-Wilk khó phát hiện vi phạm giả định và Wilcoxon chỉ trả về vài giá trị p rời rạc, nên phần suy diễn chỉ đo được mức nhất quán qua fold.
- **Chỉ là đánh giá nội bộ.** Kết quả không chứng minh hiệu quả lâm sàng, cũng không bảo đảm mô hình chạy tốt trên nhóm bệnh nhân khác.

X. Tái lập kết quả

10.1 Môi trường và cách chạy lại

Mã nguồn tại [GitHub Hepatic-Model-Comparison](https://github.com/AIVIETNAM-AIO-DoHongNhat/Hepatic-Model-Comparison), seed chung là 42, dữ liệu cuộc thi đặt trong `data/raw/`. Phiên bản thư viện chính thức ghim trong `requirements.lock.txt`; file `requirements.txt` chỉ ghi phiên bản tối thiểu nên không thay thế được. Notebook chạy theo thứ tự 01-08: 01 EDA; 02 tiền xử lý và `fold_id.csv`; 03 baseline; 04-06 ba model track; 07-08 bảng báo cáo, inference và calibration. Siêu tham số đầy đủ xem Bảng 3, phân công xác nhận nội dung xem Bảng 4.

Cài đặt và kiểm tra (các lệnh kiểm tra chỉ đọc, không ghi đè)

```
git clone https://github.com/AIVIETNAM-AIO-DoHongNhat/Hepatic-Model-Comparison.git
cd Hepatic-Model-Comparison && python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.lock.txt
python scripts/reproduce_all.py --project-root .
python scripts/run_final_analysis.py {scores|inference|calibration} --project-root .
```

Bốn lệnh kiểm tra đối chiếu môi trường hiện tại với bản đã ghim, kiểm tra 8 hash dữ liệu đầu vào, đọc kết quả cổng tái tạo và kiểm tra bảng điểm. Chế độ `--full` chạy lại notebook 02-06 rồi so 13 file với bản trước: bảng số so *theo giá trị* ở ngưỡng 10^{-12} , các file còn lại so bằng hash - chỉ nên chạy trên một bản clone sạch. Môi trường không khớp phiên bản đã ghim thì quá trình kiểm tra dừng, đó là hành vi cố ý.

10.2 Cổng tái lập

Cổng gồm ba bước chạy đúng thứ tự: (1) fit lại XGBoost trên đúng năm fold đọc từ `fold_id.csv`, đồng thời kiểm tra thứ tự lớp trả về đúng là $[C, CL, D]$ và id train/valid trong mỗi fold rời nhau hoàn toàn; (2) so 15 ô metric của bản chạy lại với `scores_track_c.csv` ở dung sai 10^{-5} rồi ghi PASS/FAIL cùng sai lệch lớn nhất ra file cổng; (3) phân tích hiệu chuẩn đọc file cổng trước tiên, thiếu file hoặc trạng thái khác PASS thì dừng, không có đường vòng.

Cổng đạt gần như tuyệt đối. Trong môi trường đã ghim, bản chạy lại tái tạo đúng `scores_track_c.csv` với sai lệch lớn nhất 5.55×10^{-17} , đạt 5/5 fold; 14/15 ô metric trùng khớp hoàn toàn, ô còn lại lệch đúng một đơn vị nhỏ nhất mà kiểu `double` biểu diễn được. Nhờ vậy dự đoán

out-of-fold dùng ở Mục 6.2 chắc chắn là cùng kết quả đã sinh ra bảng điểm, không phải một lần chạy lại gần đúng.

Sai lệch đến từ môi trường, không phải mã nguồn. Trước khi ghim phiên bản, cùng mã và cùng dữ liệu chạy trên Python 3.13.3 với XGBoost 3.3.0 cho 0/5 fold đạt cổng, sai lệch tối đa 0.0169 - lớn hơn cả chênh lệch log loss 0.024 giữa XGBoost và Random Forest. Kết quả cũ giữ trong `results/diagnostics/` và đánh dấu là đã bị thay thế. Bài học: notebook chỉ lưu tên kernel chứ không lưu phiên bản thư viện, nên phải chốt môi trường trước rồi mới đi tìm nguyên nhân chênh lệch số.

Một giới hạn phần mềm không xử lý được. Track A và Track B tái tạo giống hệt từng byte, riêng log loss của Random Forest ở fold 3 lệch 1.11×10^{-16} : `n_jobs=-1` khiến 600 cây được cộng theo thứ tự luồng do hệ điều hành quyết định, mà đổi thứ tự cộng số thực thì lệch ở chữ số cuối. Mức lệch này nhỏ hơn dung sai cổng khoảng 11 bậc độ lớn nên không ảnh hưởng kết luận nào, và cũng là lý do quy trình so theo giá trị chứ không so từng byte.

XI. Kết luận và hướng phát triển

Trong phạm vi dữ liệu và quy trình hiện tại, XGBoost là mô hình tốt nhất trong sáu ứng viên: log loss thấp nhất, macro-F1 cao nhất và xác suất được hiệu chuẩn tốt. Giá trị chính của nghiên cứu nằm ở quy trình so sánh - dùng chung fold, kiểm định có hiệu chỉnh, kiểm tra hiệu chuẩn và cổng tái tạo. Nhờ đó phân biệt được đâu là chênh lệch lặp lại đều qua các fold, đâu là chênh lệch mà thiết kế hiện tại chưa đủ sức kết luận. Mọi kết luận chỉ áp dụng cho đánh giá nội bộ và cần xem lại khi có dữ liệu đánh giá độc lập.

Ba hướng ưu tiên tiếp theo: (1) dùng cross-validation lặp lại hoặc lồng nhau để tách hẳn việc dò tham số khỏi việc đánh giá; (2) thử temperature scaling hoặc vector scaling trên một phần dữ liệu riêng, theo dõi riêng lớp CL; (3) trộn xác suất của XGBoost với Logistic Regression, trọng số chọn hoàn toàn trong tập validation. Mọi thử nghiệm mới đều phải chốt quy trình trước khi xem kết quả và ghi đầy đủ thông tin về môi trường chạy.

XII. Tài liệu tham khảo

- [1] E. R. Dickson et al., “Prognosis in primary biliary cirrhosis: Model for decision making”, *Hepatology*, vol. 10, no. 1, pp. 1-7, 1989. DOI: 10.1002/hep.1840100102
- [2] E. R. Dickson et al. “Cirrhosis patient survival prediction [dataset]”, UCI Machine Learning Repository, Accessed: Aug. 3, 2026. [Online]. Available: <https://archive.ics.uci.edu/dataset/878/cirrhosis+patient+survival+prediction+dataset-1>
- [3] L. Breiman, “Random forests”, *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001. DOI: 10.1023/A:1010933404324
- [4] T. Chen et al., “XGBoost: A scalable tree boosting system”, in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, USA: ACM, 2016, pp. 785-794. DOI: 10.1145/2939672.2939785
- [5] G. W. Brier, “Verification of forecasts expressed in terms of probability”, *Monthly Weather Review*, vol. 78, no. 1, pp. 1-3, 1950. Accessed: Aug. 3, 2026. [Online]. Available: https://journals.ametsoc.org/view/journals/mwre/78/1/1520-0493_1950_078_0001_vofeit_2_0_co_2.xml
- [6] C. Guo et al., “On calibration of modern neural networks”, in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 70, PMLR, 2017, pp. 1321-1330. Accessed: Aug. 3, 2026. [Online]. Available: <https://proceedings.mlr.press/v70/guo17a.html>
- [7] J. Demšar, “Statistical comparisons of classifiers over multiple data sets”, *Journal of Machine Learning Research*, vol. 7, pp. 1-30, 2006. Accessed: Aug. 3, 2026. [Online]. Available: <https://jmlr.org/papers/v7/demsar06a.html>

Phụ lục

A Thông tin bổ sung

1. **AI VIET NAM:** Thông tin chương trình tại aivietnam.edu.vn.
2. **GitHub Repository:** [Hepatic-Model-Comparison](#).
3. **Dataset:** ba CSV tải từ cuộc thi AIO26 Challenge trên Kaggle, đặt trong `data/raw/`. Việc phân phối lại dữ liệu phải tuân theo quy định của ban tổ chức.
4. **Model Checkpoints:** Sáu model đã fit được lưu trong `results/models/`. Kết quả chính dựa trên CV/OOF, không dựa vào một checkpoint đơn lẻ.
5. **Experiment Logs:** Điểm từng fold và runtime nằm trong thư mục `results/`: một bảng tổng hợp, ba bảng theo track và một bảng runtime.

6. **Supplementary Results:** Hình trong figures/, bảng trong tables/, OOF và manifest trong results/oof/.
7. **Presentation:** Slide hoặc video trình bày chưa được lưu trong repo tại thời điểm khóa bản báo cáo này.

B Cấu hình thí nghiệm đầy đủ

Mọi mô hình dùng seed 42 và được fit lại từ đầu trên từng fold huấn luyện. Cột “Quét” ghi các giá trị đã thử; cột “Chốt” ghi cấu hình cuối cùng, được khoá lại trước khi gộp bảng điểm. Các tham số không liệt kê giữ mặc định của thư viện.

Bảng 3: Không gian siêu tham số và cấu hình cuối của sáu mô hình.

Mô hình	Quét	Chốt
KNN	<code>n_neighbors</code> {3, 5, 7, 9, 11, 15, 21, 31, 41, 51}	$\in$ $k = 51$; uniform weights; Minkowski $p = 2$
Logistic Regression	không quét	L2, $C = 1$, lbfgs, max_iter=1000
Decision Tree	<code>max_depth</code> {2, 3, 4, 5, 6, 8, 10, 12, 15, 20, None}	$\in$ max_depth=3; Gini; min_samples_leaf=1
Naive Bayes	không quét	GaussianNB, var_smoothing= 10^{-9}
Random Forest	<code>n_estimators</code> $\in$ {200, 400, 600}; <code>max_depth</code> $\in$ {6, 8, 10, 14, None}	600 cây, max_depth=14, n_jobs=-1
XGBoost	<code>n_estimators</code> $\in$ {200, 400}; <code>max_depth</code> 2-6; <code>learning_rate</code> $\in$ {0.03, 0.05, 0.10, 0.15, 0.20}	400 cây, max_depth=2, lr=0.15, eval_metric=mlogloss

Môi trường chính thức (requirements.lock.txt, 160 package): Python 3.9.18, scikit-learn 1.6.1, XGBoost 2.1.4, NumPy 1.26.4, pandas 2.3.3, SciPy 1.13.1, Matplotlib 3.9.2.

C Nguồn của từng kết quả

- Hiệu năng: results/scores_all.csv, tables/report_model_comparison.csv.
- Kiểm định: tables/report_significance.csv, tables/assumption_checks.csv.
- Hiệu chuẩn: results/oof/; trong tables/ có calibration_metrics.csv và calibration_ece_bin_sensitivity.csv.
- Ablation: trong tables/ có knn_scaling_ablation.csv và tree_vs_ensemble.csv.
- Môi trường và tái lập: requirements.lock.txt; trong results/oof/ có manifest, kết quả cổng tái tạo và metric từng fold; bản chẩn đoán cũ đã bị thay thế nằm trong results/diagnostics/.

D Phân công xác nhận nội dung

Bảng 4: Bảng phân công công việc nhóm

Thành viên	Nhiệm vụ
Trương Trung Tín	Dữ liệu, tiền xử lý, folds, Random Forest, XGBoost
Lê Đức Tùng Dương	KNN, Logistic Regression, ablation scaling
Đỗ Hồng Nhất	Decision Tree, Naive Bayes, cây đơn-ensemble
Đinh Văn Anh Khôi	Kiểm định, khoảng tin cậy, hiệu chỉnh, calibration
Nguyễn Thị Cẩm Vân	Hình, bảng, tích hợp báo cáo và kiểm soát chất lượng